The Generous Auction

A geometric pay-as-bid tick book, worked through by hand

This note explains the distribution rule proposed for the MONO auction: how one round's emission is split across the order book, why the split has the shape it does, and how the whole thing collapses into a single number that a contract can read in constant time. Every number in the worked examples was recomputed from scratch rather than copied from the spec.

> **Status.** The rule below is implemented twice: `src/GenerousAuction.sol` for a single sale, and `src/MonoAuction.sol` for the same mechanism behind a multi-market wrapper. Both solve each round in one sweep over a bounded price window instead of carrying the lazy accumulator of §6. §1.1 describes the strict-priority rule they replaced; `index.html` is an interactive model of both.

1 The book

The auction sells a fixed emission of tokens per round in exchange for a currency. Bids are placed on a ladder of discrete price levels called *ticks*. Tick i carries a price p_i, and the ladder is geometric in price:

$$p_i \;=\; \text{floor} \cdot (1+\delta)^i.$$

A bidder does not name a quantity. They escrow a *budget* b_i of currency at a tick, and that budget stands there, round after round, until it is spent or withdrawn. The tick's budget is the sum of everyone standing on it.

Two properties are fixed. Both predate this design and neither is touched by it:

- **Pay-as-bid.** You buy at *your own* tick's price, not at a common clearing price. Sales are final.
- **Pro-rata within a tick.** Whatever a tick spends in a round is divided among the bids standing on it in proportion to their budgets.

Everything that follows concerns exactly one question: *how is a round's emission R divided **between** ticks?*

1.1 The current answer, and its problem

Today the rule is strict priority. Fill the highest tick completely, then the next, then the next, until R runs out. Whichever tick the emission happens to stop inside is the *marginal* tick; it fills partially, and everything below it gets nothing.

That is simple, and it is what every clearing auction does. It also produces a cliff. A bidder one tick above the margin fills completely; a bidder one tick below gets nothing, this round and possibly for many more. The outcome turns entirely on which side of the line you land on, and the line moves with demand you cannot see.

The generous auction replaces that cliff with a gradient.

2 The rule

Let A be the set of *live* ticks — those with budget remaining — and let

$$\tau \;=\; \max A$$

be the highest live tick, the top of the book. Give every live tick a weight that decays geometrically with its distance below the top:

$$\boxed{w_i \;=\; q^{\tau - i}} \qquad q \in (0, 1].$$

The top tick weighs 1. Each step down multiplies by q. Normalising by $W = \sum_{j \in A} w_j$, tick i takes a share

$$s_i \;=\; \frac{w_i}{W}$$

of the round's emission: $a_i = R\, s_i$ tokens, costing $c_i = a_i\, p_i$ of its budget.

So every live tick buys every round. Not equally — a tick four levels down at $q = 0.5$ buys at one-sixteenth the rate of the top — but continuously, with no threshold anywhere.

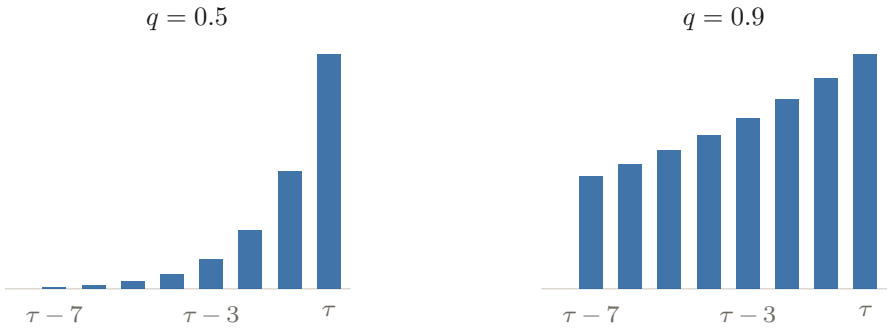

Figure 1: The weight ladder $w = q^{\tau - i}$. Small q concentrates the emission at the top of the book; large q spreads it far down. The knob is continuous.

2.1 The waterfall

A tick's budget is finite, so a tick can run out *in the middle* of a round's emission. When it does, it leaves A, and the remainder of that same emission is redistributed by recomputed weights among the survivors. If a second tick exhausts, redistribute again. There are at most as many redistributions as there are exhausted ticks.

This is how unfilled emission moves upward, and it is not a separate mechanism. It is what renormalisation does on its own: when a tick leaves, W shrinks, so every surviving share w_i/W grows — most of all at the top, which carries the most weight. Nothing is transferred from the exhausted tick. The shares simply rescale.

If every tick exhausts, the leftover is recorded as `unsold`. Under the strict setting this holds even when a single bidder is alone in the book: they still receive only their own share, which is the round-level guarantee against a whale simply buying the emission.

3 Worked example: four rounds, one exhaustion

Take floor $= 1$, $\delta = 5\%$, and three live ticks. Let $q = 0.5$ and $R = 70$ tokens per round.

	price p_i	budget b_i	weight w_i	share s_i	tokens/round
tick 2 (τ)	1.1025	100	1	$4/7 = 57.1\%$	40
tick 1	1.0500	210	0.5	$2/7 = 28.6\%$	20
tick 0	1.0000	500	0.25	$1/7 = 14.3\%$	10
		$W = 1.75$			$R = 70$

Rounds 1–2. Nothing exhausts, so the split is constant and both rounds are identical. Per round, tick 2 takes 40 tokens for 44.10 of currency, tick 1 takes 20 for 21.00, and tick 0 takes 10 for 10.00 — so *over the two rounds together* tick 2 has taken 80 tokens for 88.20, leaving it 11.80; tick 1 has taken 40 for 42.00 and tick 0 20 for 20.00.

	share	tokens/rd	currency/rd	budget after R1	after R2
tick 2 (τ)	4/7	40	44.10	55.90	11.80
tick 1	2/7	20	21.00	189.00	168.00
tick 0	1/7	10	10.00	490.00	480.00
Σ		70	75.10		

Tick 2 is the one to watch: it burns 44.10 a round against a budget of 100, so it cannot survive a third.

Round 3 — the interesting one. Tick 2 has 11.80 left, which buys $11.80/1.1025 = 10.70$ tokens. At a 4/7 share it reaches that after only

$$10.70 \times \tfrac{7}{4} \;=\; 18.73 \text{ tokens of the } 70 \qquad (26.8\% \text{ of the round}),$$

at which point its budget is gone. On that first slice tick 1 receives 5.35 and tick 0 receives 2.68.

Now tick 2 is out. The top becomes $\tau = 1$, weights rebuild as $w_1 = 1,\ w_0 = 0.5$, so $W = 1.5$ and the shares are 2/3 and 1/3. The remaining 51.27 tokens pour by *those*: tick 1 takes 34.18, tick 0 takes 17.09.

Tick 1's share jumped from 28.6% to 66.7% the instant tick 2 exhausted — mid-round, without any explicit transfer.

	stage 1 — $W = 1.75$		stage 2 — $W = 1.5$		round 3		
	share	tokens	share	tokens	tokens	currency	budget left
tick 2	4/7	10.70	exhausted		10.70	11.80	0.00
tick 1	2/7	5.35	2/3	34.18	39.53	41.51	126.49
tick 0	1/7	2.68	1/3	17.09	19.77	19.77	460.23
Σ		18.73		51.27	70.00	73.08	

The two stages are one emission, not two: $18.73 + 51.27 = 70$. Only the shares changed partway through.

Round 4. Two ticks left, shares 2/3 and 1/3: tick 1 takes 46.67 tokens, tick 0 takes 23.33.

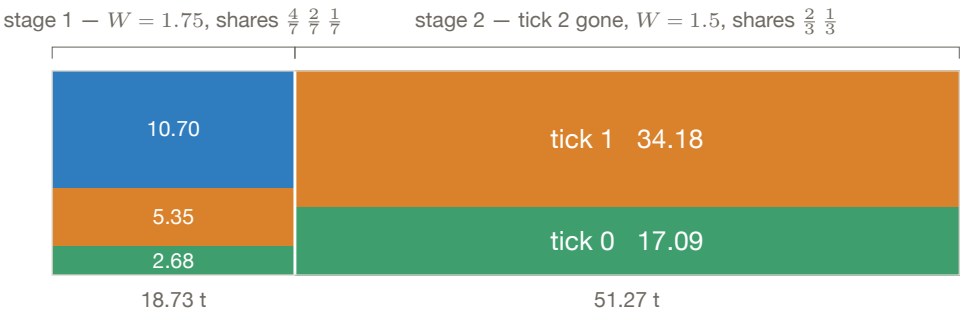

Figure 2: Round 3 of the example: the emission of 70 tokens, drawn left to right. Tick 2 (■) exhausts 26.8% of the way through; from that point the same round's remainder flows by renormalised weights, and tick 1's rate more than doubles.

	share	tokens	currency	budget left
tick 2 (gone)	—	—	—	0.00
tick 1 (τ)	$2/3$	46.67	49.00	77.49
tick 0	$1/3$	23.33	23.33	436.90
Σ		70.00	72.33	

No renormalisation this round — the book did not change, so the shares from stage 2 of round 3 simply carry over.

Totals over four rounds.

	currency spent	tokens bought	check
tick 2	100.00 (all of it)	90.70	$100/1.1025 = 90.70$
tick 1	132.51	126.20	77.49 of budget still standing
tick 0	63.10	63.10	price is 1.0
Σ	295.61	**280.00**	$= 4R$ ✓

Tick 1 and tick 0 are still standing with budget left, and they carry on into round 5 against whoever bids next. Nothing was migrated, re-keyed, or refunded.

4 Why geometric

The decay could have been any decreasing function of distance. Two properties single out q^d, and both are load-bearing.

4.1 Shift-invariance

Because

$$\frac{q^{d+m}}{\sum_j q^{d_j+m}} = \frac{q^d}{\sum_j q^{d_j}},$$

a new tick appearing *above* the current top does not change the relative shares of anyone already in the book. Everyone's weight is multiplied by the same q^m, which cancels.

This kills the obvious manipulation. Under a rule without this property, a nominal bid parked far above the market stretches the curve and starves everyone below. Here it does nothing to anyone else: the manipulator becomes the top tick and pays the top price for the privilege. A hyperbolic weight $1/(1+d)$ does not have this property.

4.2 Factorisability

$$w_i \;=\; q^{\tau-i} \;=\; q^{\tau}\cdot q^{-i}$$

splits into a purely global factor and a purely per-tick factor. This is what makes the whole scheme computable lazily, and it is the subject of §6. With a hyperbolic weight no such split exists, and reading a single bidder's position degrades to walking the entire history of top-of-book changes.

> One consequence to be clear about: distance is measured in **price ticks**, not in rank among live ticks. A tick a hundred levels below the top receives $q^{100} \approx 0$ even if it happens to be the second-highest bid in the book. That is deliberate — the window of participation is defined by distance in price, not by queue position. Rank would also destroy laziness, since your rank changes when strangers bid.

5 Worked example: exhaustion order is not price order

This example is short, and it is the one that dictates the data structure a contract needs.

The column that does the dictating is the last, the key κ_i. Track the draw with one scalar C, the *accumulator*, every live tick holding $q^{-i}\cdot C$ tokens. A tick takes its share until its escrow is spent, then drops out — at a fixed level of C, known before a token is poured. That level is κ_i, so one sorted pass solves the whole waterfall.

Four live ticks one grid step apart, $q = 0.5$, and a draw of $R = 150$ tokens. Each tick holds escrow b_i (currency) at price p_i (currency per token), so its *capacity* in tokens is $\text{cap}_i = b_i/p_i$:

	escrow b_i	price p_i	capacity cap_i	q^{-i}	opening share	key κ_i
tick 3 (τ)	80	4	20	8	$8/15 = 53.3\%$	2.5
tick 2	600	3	200	4	$4/15 = 26.7\%$	50
tick 1	20	2	10	2	$2/15 = 13.3\%$	5
tick 0	100	1	100	1	$1/15 = 6.7\%$	100
			$V = \sum q^{-j} = 15$			

Column by column, with tick 3 as the worked instance:

- **escrow** b_i — currency standing at the tick. An input.
- **price** p_i — currency per token there. An input; the four are one step apart.
- **capacity** $\text{cap}_i = b_i/p_i$ — most tokens that escrow can buy. Tick 3: $80/4 = 20$.
- **weight** q^{-i} — pull on each token poured; doubles per level up at $q = 0.5$. Tick 3: $0.5^{-3} = 8$.
- **opening share** q^{-i}/V — weight over the live total $V = \sum_j q^{-j} = 15$. Tick 3: $8/15 = 53.3\%$.
- **key** $\kappa_i = \text{cap}_i \cdot q^{i}$ — capacity per unit of weight, i.e. cap_i/q^{-i}: the level C must reach for tick i to exhaust. Tick 3: $20/8 = 2.5$.

Neither middle column is an allocation. **Capacity** is a ceiling: the most a tick could ever take, if the draw were unlimited. **Opening share** is a rate: the fraction of each marginal token a tick absorbs *while every tick is still alive*. Tick 3's 53.3% holds only for the first 37.5 tokens, after which it is full and its share is redivided among the survivors. The allocations themselves appear below, after the sweep.

The key is $\kappa_i = \text{cap}_i \cdot q^i$, and a tick exhausts exactly when the accumulator reaches its key. Sorting by κ gives the order in which ticks die:

$$\text{tick } 3\ (2.5) \ \rightarrow\ \text{tick } 1\ (5) \ \rightarrow\ \text{tick } 2\ (50) \ \rightarrow\ \text{tick } 0\ (100).$$

Tick 1 exhausts before tick 2, although tick 2 is priced higher and draws twice the share. Tick 1 is a small budget on a good level; tick 2 is a large budget one level down. A rich low tick outlives a poor high tick. Exhaustion order and price order are different orderings, and the contract must maintain the first one explicitly — the sorted structure the current implementation gets for free from price is no longer sufficient.

Running the draw, tracking a single scalar C with $\Delta T = V \cdot \Delta C$:

stage	event	tokens poured	C after	V after
1	tick 3 exhausts at $\kappa = 2.5$	37.5	2.5	7
2	tick 1 exhausts at $\kappa = 5$	17.5	5	5
3	emission runs out first	95	24	5

Final allocation: tick 3 takes 20 (its whole capacity), tick 1 takes 10 (likewise), tick 2 takes $q^{-2} \cdot C = 4 \times 24 = 96$, tick 0 takes $1 \times 24 = 24$. The four sum to 150 exactly.

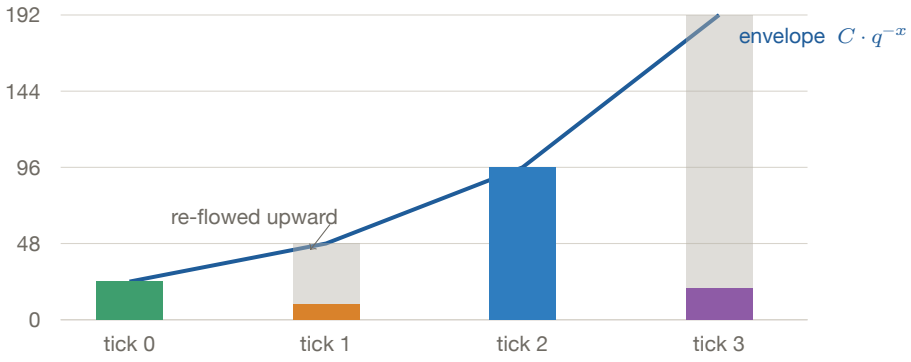

Figure 3: The same draw, seen as one curve. Every tick that survives sits *exactly* on the envelope $C \cdot q^{-x}$ — check $96/24 = 4 = q^{-2}$. Every tick that exhausted sits below it, and the shaded gap above such a bar is precisely the emission that re-flowed to the survivors.

That single observation — survivors lie on one geometric curve, and only the exhausted ticks depart from it — is the whole reason the next section works.

6 The accumulator: why this is cheap

Read naively, every tick's state depends on everything consumed above it, across all history. That is unaffordable on chain. Factorisability removes the dependence.

Between events the live set, the top τ and W are constant, so tick i drains at a constant rate $R\,q^{\tau-i}/W$. Define one global accumulator advancing at

$$\mathrm{d}G \;=\; \frac{\mathrm{d}T}{R\,W},$$

where T is cumulative emission. Then over *any* interval,

$$\boxed{\text{tokens taken by tick } i \;=\; q^{\tau-i}\cdot R\cdot \Delta G}$$

One scalar describes every live tick simultaneously. Reading a tick costs one exponentiation and no iteration — no per-bidder loop, no history walk, no off-chain indexer. A tick that nobody touches costs nothing at all.

The waterfall of §3 is *not* a second algorithm bolted on. It is the same G advancing at piecewise-constant speed, and the speed changes whenever W changes. That buys a property worth having: the result is **path-independent**. A single draw of 280 tokens produces exactly the outcome of four rounds of 70.

6.1 The example, recomputed from one number

Take the four rounds of §3 again and track only G. When the top of book moves from τ to τ', everything held in "units of the top" is rescaled by $q^{\tau-\tau'}$:

	segment	ΔG	G after
rounds 1–2	140 t at $W = 1.75$	$+1.1429$	1.1429
round 3a	18.73 t at $W = 1.75$	$+0.1529$	1.2958
	tick 2 exhausts; top moves $2\to1$*; rescale* $\times q$		0.6479
round 3b	51.27 t at $W = 1.5$	$+0.4883$	1.1362
round 4	70 t at $W = 1.5$	$+0.6667$	**1.8028**

Tick 2's exhaustion key was predictable in advance: $100/(70\times1.1025\times1) = 1.2958$ — precisely where G arrived. And now read the two survivors straight off the final G, with $\tau = 1$:

$$a_0 = R\,q^{1-0}\,G = 70\times0.5\times1.8028 = 63.10, \qquad a_1 = R\,q^{0}\,G = 70\times1.8028 = 126.20.$$

Both match the round-by-round totals in §3 exactly. Four rounds, one mid-round exhaustion and one rescale, recovered from a single stored scalar and one exponentiation each.

6.2 What that costs in practice

Three pieces of machinery pay for it:

- **A rescale on top-of-book changes.** G, W and the exhaustion keys are all carried in units of the current top; when the top moves they all scale by the same $q^{\tau-\tau'}$. Because the factor is common, the *ordering* of the keys is untouched — so the rescale is a constant-time operation, never a sweep.
- **An ordered structure of exhaustion keys**, for the reason established in §5. A tick's key changes whenever its budget does, so every bid and every withdrawal has to re-file it.

- **Aggregate sums for revenue.** Carrying $W_p = \sum p_i q^{\tau - i}$ alongside W gives revenue and tokens sold per segment without visiting any tick.

Worth keeping in proportion: the strict-priority rule of §1.1 is the $q \to 0$ limit of this scheme. Both run the same number of loop iterations — one per exhausted tick, and each tick exhausts once. The extra cost is a constant factor inside the loop, not a change in asymptotics.

7 The knob

q interpolates continuously between the two degenerate rules:

$q \to 0$	strict price priority — the current mechanism
$q = 1$	equal split across every live tick, price ignored

In a dense book the top tick's share is

$$\frac{1-q}{1-q^n} \longrightarrow 1-q,$$

so q sets a soft ceiling on the top of the book: $q = 0.75$ caps it near 25% of each round, $q = 0.9$ near 10%. The participation window runs roughly $\log_q(\text{dust})$ ticks down — at $q = 0.9$, several hundred levels; at $q = 0.5$, about sixty.

Note the qualifier: that ceiling holds only when the book is dense beneath the top. A hard cap under *any* book shape requires a separate overlay and is not provided by q alone.

The incentive to bid higher stays continuous throughout. Moving up one tick multiplies your fill rate by $1/q$ in exchange for a price one tick worse — a smooth trade of price against urgency, rather than a bet on where a threshold will land.

8 Properties and limits

No clearing price, twice over. As in any pay-as-bid design there is no single price at which everyone transacts. The geometric variant also dissolves the sharp marginal boundary, so there is not even a "last filled tick" to quote. Where a reference price is needed downstream, the two available ones are the top of book τ and the realised average raised/sold.

Splitting a bid. A bidder who fragments one budget across k ticks collects k weights instead of one. The friction against this is the minimum bid size times k, plus the genuinely worse fill rate of every level below their best. Whether that friction suffices is an open calibration question; a concave budget-weight or an explicit cap would close it. Bidding from several addresses cannot be handled inside the mechanism at all — that needs a validation hook.

Anchoring. Dead, by shift-invariance (§4.1).

Novelty is the risk. Each component here is proven in production elsewhere: virtual-time accumulators with a sorted event queue are how fair-queueing network schedulers and the Linux CFS scheduler work; lazy $O(1)$ reads through a global index are standard in staking-reward and lending contracts; the capped waterfall with renormalisation is the water-filling construction from information theory and the classical bankruptcy problem.

The *combination* is what has not been run in production. Classical matching engines never split volume between price levels; this design does so deliberately. The economics are therefore unproven

in a way the engineering is not, which is why simulation against realistic books — and specifically against an adversarial whale — should precede deployment rather than follow it.

Every number in §3 and §5 was recomputed from scratch rather than copied from the spec. The model in `index.html` implements the same rule; open it with `?selftest` to check it against both examples.